

AI휴먼 과정

7일차 | Python 프로그래밍

강의주제: 파일 입출력 · 예외 처리 · 모듈과 import

오늘의 핵심 질문

프로그램이 데이터를 파일에 저장하고, 오류 상황에서도 안전하게 계속 동작하게 하려면 어떻게 해야 할까요?

오늘의 산출물

안전한 파일 처리 프로그램



오늘의 학습목표

수업이 끝나면 다음 작업을 직접 구현할 수 있습니다.

- ☐ 텍스트 파일을 생성하고 내용을 저장할 수 있습니다.
- ☐ 파일 전체 또는 여러 줄을 읽을 수 있습니다.
- ☐ `with open()`을 사용해 파일을 안전하게 열고 닫을 수 있습니다.
- ☐ `try-except`로 파일 오류와 값 변환 오류를 처리할 수 있습니다.
- ☐ `else, finally`의 실행 시점을 설명할 수 있습니다.
- ☐ 표준 라이브러리의 모듈을 `import`하여 기능을 재사용할 수 있습니다.
- ☐ `csv` 모듈로 `CSV` 파일을 읽고 쓸 수 있습니다.
- ☐ 파일 처리 로직을 함수로 분리하여 유지보수하기 쉬운 프로그램을 만들 수 있습니다.

6일차 복습: 함수와 오늘 내용의 연결

6일차 핵심 흐름

함수(Function)

특정 작업을 이름 붙여 재사용하는 코드 단위

매개변수(Parameter)

함수가 받을 값을 정의하는 변수

인수(Argument)

함수를 호출할 때 실제로 전달하는 값

반환값(Return Value)

함수 실행 결과를 호출한 곳으로 돌려주는 값

오늘은 함수를 파일 처리와 결합합니다.

```
def read_text(path):  
    with open(path, "r", encoding="utf-8") as file:  
        return file.read()
```

핵심 관점

파일을 여는 코드, 읽는 코드, 오류를 처리하는 코드를 각각 역할별로 나누면 프로그램을 이해하고 수정하기 쉬워집니다.

파일(File)이 필요한 이유

프로그램의 변수는 실행이 끝나면 사라집니다.

예시

```
name = "홍길동"  
score = 85
```

프로그램 종료 후에도 값을 남기려면 파일이나 데이터베이스 같은 저장 공간이 필요합니다.

파일 사용 사례

- 학습 기록 저장
- 고객 문의 로그 저장
- CSV 데이터 분석
- 설정값 저장
- 실행 결과 리포트 저장
- AI 서비스의 입력/출력 기록 저장

핵심 개념

- 메모리: 실행 중 임시 저장
- 파일: 실행이 끝나도 남는 영구 저장

File I/O란?

File I/O = File Input / Output = 파일 입출력

Input

파일의 데이터를 프로그램으로 읽어오는 것

Output

프로그램의 데이터를 파일에 저장하는 것

전체 흐름



대표 함수

```
open("파일경로", "모드", encoding="utf-8")
```

파일 객체를 통해 `read()`, `write()` 등의 작업을 수행합니다.

open() 함수의 기본 구조

```
file = open("memo.txt", "r", encoding="utf-8")  
content = file.read()  
file.close()
```

각 요소의 의미

"memo.txt"

열 파일의 경로

"r"

읽기 모드(Read)

encoding="utf-8"

문자 인코딩 지정

file.read()

파일 내용 읽기

file.close()

파일 닫기

⚠️ 주의점: 파일을 열었다면 닫아야 합니다. 예외가 발생하면 `close()`가 실행되지 않을 수 있습니다. 실무에서는 `with open()` 사용을 권장합니다.

파일 열기 모드 정리

모드	영문 의미	파일이 없을 때	기존 내용	설명
r	read	오류	유지	읽기
w	write	생성	덮어씀	새로 쓰기
a	append	생성	뒤에 추가	이어 쓰기
x	exclusive create	생성	이미 있으면 오류	새 파일 생성

예시

```
open("memo.txt", "r", encoding="utf-8")
open("memo.txt", "w", encoding="utf-8")
open("memo.txt", "a", encoding="utf-8")
```

⚠ w 모드는 기존 내용을 지울 수 있으므로 파일 경로와 목적을 확인해야 합니다.

with open()을 사용하는 이유

권장 방식

```
with open("memo.txt", "r", encoding="utf-8") as file:  
    content = file.read()  
    print(content)
```

자동 닫기

블록이 끝나면 파일을 자동으로 닫습니다.

간결한 코드

코드가 간결합니다.

안전한 자원 정리

오류가 발생해도 자원 정리가 안전합니다.

개념

- with: 자원을 안전하게 관리하는 컨텍스트 관리자(Context Manager) 문법
- as file: 열린 파일 객체를 file이라는 변수로 사용

파일 쓰기: write()

```
message = "AI휴먼 7일차 파일 입출력 학습"  
with open("memo.txt", "w", encoding="utf-8") as file:  
    file.write(message)
```

특징

- write()는 문자열을 저장합니다.
- 줄바꿈이 필요하면 \n을 직접 넣습니다.
- 숫자는 문자열로 변환해야 합니다.

```
score = 90  
file.write(str(score))
```

여러 줄 저장

```
file.write("이름: 홍길동\n")  
file.write("점수: 90\n")
```

파일 이어 쓰기: append 모드

기존 기록 뒤에 새로운 내용을 추가할 때 **a** 모드를 사용합니다.

```
with open("study_log.txt", "a", encoding="utf-8") as file:  
    file.write("7일차: 파일 입출력 학습 완료\n")
```

활용 사례

- 학습 일지
- 프로그램 실행 로그
- 주문 처리 기록
- 오류 기록

w와 a 비교

- **w**: 새 문서처럼 다시 작성
- **a**: 기존 문서 뒤에 계속 기록

파일 읽기: read(), readline(), readlines()

전체 읽기

```
content = file.read()
```

한 줄 읽기

```
line = file.readline()
```

모든 줄을 리스트로 읽기

```
lines = file.readlines()
```

예시 결과

```
["홍길동,90\n", "김영희,85\n"]
```

선택 기준

- 작은 파일 전체 처리: read()
- 한 줄씩 단계 처리: readline() 또는 반복문
- 줄 단위 리스트 필요: readlines()

파일 객체를 반복문으로 읽기

큰 파일은 한 번에 모두 메모리에 올리지 않고 한 줄씩 처리할 수 있습니다.

```
with open("scores.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

strip()의 역할

- 문자열 앞뒤의 공백 제거
- 줄 끝의 \n 제거에 자주 사용

장점

- 코드가 단순합니다.
- 큰 파일 처리 시 메모리 사용량을 줄일 수 있습니다.

파일 경로(Path)의 이해

상대 경로

```
./data/memo.txt
```

현재 프로그램 위치를 기준으로 찾습니다.

절대 경로 예시

```
C:/workspace/project/data/memo.txt
```

핵심 용어

Path

파일 또는 폴더의 위치

pathlib 사용

Python에서는 pathlib을 사용하면 운영체제에 따른 경로 차이를 줄일 수 있습니다.

```
from pathlib import Path
path = Path("data") / "memo.txt"
print(path)
```

Directory

폴더

Relative Path

상대 경로

Absolute Path

절대 경로

Exception이란?

Exception = 예외

프로그램 실행 중 예상하지 못한 문제가 발생한 상태입니다.

대표 예외

FileNotFoundError

파일이 없음

ValueError

값의 형식이 잘못됨

ZeroDivisionError

0으로 나눔

PermissionError

파일 접근 권한 문제

UnicodeDecodeError

문자 인코딩 문제

예외 처리 목적

오류를 숨기는 것이 아니라, 예상 가능한 문제에 대해 사용자가 이해할 수 있는 대응을 제공하는 것입니다.

try-except 기본 구조

```
try:
    # 오류가 발생할 수 있는 코드
    with open("memo.txt", "r", encoding="utf-8") as file:
        print(file.read())
except FileNotFoundError:
    # 해당 오류가 발생했을 때 실행
    print("파일을 찾을 수 없습니다.")
```

실행 흐름

01

try 블록 실행

03

지정한 예외가 발생하면 해당 **except** 실행

02

오류가 없으면 **except**를 건너뛴

04

프로그램이 다음 코드로 계속 진행

여러 예외를 구분해서 처리하기

```
try:
    text = "abc"
    number = int(text)
except ValueError:
    print("숫자로 변환할 수 없습니다.")
```

파일 처리 예시

```
try:
    with open("data.txt", "r", encoding="utf-8") as file:
        value = int(file.read())
except FileNotFoundError:
    print("파일이 없습니다.")
except ValueError:
    print("파일 내용이 숫자가 아닙니다.")
```

원칙

- 예상 가능한 예외는 구체적인 타입으로 처리합니다.
- 모든 오류를 무조건 **except:** 하나로 처리하면 원인 파악이 어려워집니다.

else와 finally

```
try:
    number = int("100")
except ValueError:
    print("변환 실패")
else:
    print("변환 성공:", number)
finally:
    print("작업 종료")
```

try

먼저 실행

except

예외 발생 시 실행

else

예외가 발생하지 않았을 때 실행

finally

예외 발생 여부와 관계없이 마지막에 실행

활용

- 성공 메시지 출력
- 로그 기록
- 연결/자원 정리

예외 객체의 메시지 확인

오류 원인을 기록해야 할 때 예외 객체를 받을 수 있습니다.

```
try:
    with open("unknown.txt", "r", encoding="utf-8") as file:
        print(file.read())
except FileNotFoundError as error:
    print("오류:", error)
```

as error의 의미

- 발생한 예외 객체를 변수에 저장
- 실제 오류 메시지를 확인 가능

주의

사용자에게 보여주는 메시지와 개발자가 확인할 상세 로그는 분리하는 것이 좋습니다.

Module이란?

Module = 모듈

Python 코드를 기능별로 저장한 하나의 .py 파일입니다.

예시

```
main.py  
file_utils.py
```

file_utils.py

```
def read_text(path):  
    ...
```

main.py

```
import file_utils  
text = file_utils.read_text("memo.txt")
```

효과

- 코드 재사용
- 역할 분리
- 유지보수 향상
- 협업 시 파일 단위 업무 분담 가능

import 사용 방법

모듈 전체 가져오기

```
import csv
```

특정 기능 가져오기

```
from pathlib import Path
```

별칭 사용

```
import datetime as dt
```

사용 예

```
from pathlib import Path  
path = Path("data") / "scores.csv"
```

용어

- Standard Library: Python 설치 시 기본 제공되는 표준 라이브러리
- Package: 여러 모듈을 묶은 구조
- import: 다른 모듈의 코드를 현재 프로그램에서 사용하도록 가져오는 문법

CSV 파일과 csv 모듈

CSV = Comma-Separated Values

쉼표 등 구분자로 데이터를 행과 열 형태로 저장하는 텍스트 형식입니다.

예시

```
name, score
```

```
홍길동, 90
```

```
김영희, 85
```

Python 표준 라이브러리

```
import csv
```

장점

- 별도 설치가 필요 없습니다.
- 엑셀, 스프레드시트, 데이터 분석 도구와 교환하기 좋습니다.
- 이후 Pandas 학습의 기초가 됩니다.



CSV 쓰기과 읽기

CSV 쓰기

```
import csv
with open("scores.csv", "w", newline="",
encoding="utf-8-sig") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "score"])
    writer.writerow(["홍길동", 90])
```

CSV 읽기

```
with open("scores.csv", "r", encoding="utf-8-sig") as
file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

`newline=""`

CSV 저장 시 불필요한 빈 줄 문제를 줄이는 데 사용됩니다. (Windows OS에서 발생)

실무형 구조: 파일 처리 기능을 함수로 분리

한 함수에 모든 코드를 넣지 않습니다.

```
def save_text(path, text):  
    ...  
def read_text(path):  
    ...  
def load_scores(path):  
    ...
```

역할 분리의 장점

- 수정 범위가 작아집니다.
- 테스트하기 쉽습니다.
- 중복 코드가 줄어듭니다.
- 다른 프로젝트에서 재사용하기 쉽습니다.

오늘의 설계 원칙

하나의 함수는 하나의 주요 역할에 집중합니다.

의사코드: 안전한 파일 읽기 프로그램

Pseudocode = 의사코드

실제 프로그래밍 언어 문법보다 처리 절차에 집중해서 작성합니다.

```
프로그램 시작
파일 경로를 준비한다
시도한다
    파일을 읽기 모드로 연다
    파일 내용을 읽는다
    내용을 화면에 출력한다
파일이 없으면
    사용자가 이해할 수 있는 오류 메시지를 출력한다
문자 인코딩 오류가 발생하면
    인코딩을 확인하라는 메시지를 출력한다
마지막으로
    작업 종료 메시지를 출력한다
프로그램 종료
```

의사코드 작성 목적

- 구현 전에 흐름을 점검
- 팀원과 로직 공유
- 복잡한 문법보다 문제 해결 순서에 집중

종합 실습: 안전한 파일 처리 프로그램

실습 흐름

01

day7_data 폴더 만들기

03

기존 파일에 학습 기록 추가하기

05

존재하지 않는 파일 읽기 시도

07

문자열을 숫자로 변환하면서 ValueError 처리

09

CSV 파일 읽기

11

파일 처리 로직을 함수로 분리

02

텍스트 파일 새로 저장하기

04

파일 전체와 줄 단위 읽기

06

FileNotFoundError 처리

08

CSV 파일 저장

10

점수 평균 계산

12

표준 모듈 csv, pathlib 사용

📌 오늘의 핵심 산출물: 오류가 발생해도 이유를 안내하고 다음 작업을 계속 수행하는 파일 처리 프로그램

오늘의 테스트·과제·리뷰

단답형 테스트

- File I/O 핵심 용어
- r, w, a 모드 차이
- with open()의 목적
- try-except-else-finally 실행 흐름
- Module과 import
- CSV 처리 기초

과제 단계

- Basic: 텍스트 학습 기록 저장·읽기
- Application: CSV 성적 파일을 안전하게 읽고 통계 계산
- Challenge: CSV 처리 기능을 함수와 모듈 구조로 분리하고 오류 메시지 개선

❑ 오늘 반드시 설명할 수 있어야 하는 문장: "파일 입출력은 프로그램 밖에 데이터를 저장하고 다시 읽는 기능이며, 예외 처리는 예상 가능한 오류 상황에서도 프로그램이 안전하게 대응하도록 만드는 방법입니다."

다음 학습 예고

클래스(Class)

객체(Object)

인스턴스(Instance)

속성(Attribute)

메서드(Method)

지금까지의 Python 문법을 하나의 프로그램으로 통합